

Escuela Superior Politécnica de Chimborazo

Facultad de Informática y Electrónica, Ingeniería de Software



Lista LECTURA COMPLEMENTARIA: Ejecución de pruebas simbólicas

Kevin Gerardo Quilligana Vivas - 7462

VERIFICACIÓN Y VALIDACIÓN DE SOFTWARE

Riobamba, 08 de mayo del 2025

1 Introducción

En el desarrollo de software moderno, garantizar la corrección del código ante cualquier combinación posible de entradas es un desafío que las pruebas tradicionales no pueden resolver de forma exhaustiva. Una suite de pruebas unitarias cubre únicamente los casos que el equipo elige manualmente; cualquier valor de borde o combinación de parámetros omitida puede ocultar un error que solo aparecerá en producción.

Ante este panorama, las pruebas simbólicas emergen como una técnica poderosa que permite explorar sistemáticamente todos los caminos de ejecución de un programa, razonando sobre rangos completos de entradas en lugar de valores puntuales.

La ejecución simbólica, técnica en la que se basa este tipo de pruebas, fue propuesta originalmente por King (1976) y ha experimentado un renacimiento significativo gracias al avance de los solvers SMT (Satisfiability Modulo Theories).

Herramientas como KLEE, CBMC y Angr han demostrado que esta técnica no solo es teóricamente atractiva sino prácticamente efectiva para detectar errores en software real, desde sistemas operativos hasta contratos inteligentes en blockchain.

1.1 Objetivo

1.1.1 *Objetivo general*

Analizar el papel de las pruebas simbólicas en la ingeniería de software, describiendo su funcionamiento técnico, sus herramientas representativas y su contribución a la calidad del software.

1.1.2 *Objetivos específicos*

- Explicar los fundamentos conceptuales y operativos de la ejecución simbólica.
- Describir cómo se construyen las condiciones de camino y cómo se resuelven mediante solvers SMT.
- Comparar las pruebas simbólicas con las pruebas tradicionales.
- Identificar las herramientas más relevantes y sus características principales.
- Analizar los tipos de software donde las pruebas simbólicas aportan mayor valor.
- Aplicar el razonamiento simbólico a ejemplos prácticos.

2 Definición Fundamentos de Pruebas Simbólicas

Las pruebas simbólicas son una técnica de análisis de software en la que las entradas de un programa se representan mediante variables simbólicas en lugar de valores específicos. Durante la ejecución, el sistema analiza las operaciones realizadas sobre dichas variables y construye restricciones lógicas llamadas condiciones de camino. Cada condición de camino representa los requisitos necesarios para que el programa siga una determinada ruta de ejecución. Posteriormente, un *solver* SMT evalúa si dichas restricciones son satisfacibles y genera valores concretos que permiten ejecutar cada camino identificado. Este enfoque permite incrementar significativamente la cobertura de pruebas y detectar errores relacionados con rutas de ejecución poco comunes o difíciles de reproducir manualmente.

2.1 Componentes de una ACL

Criterio	Pruebas Tradicionales	Pruebas Simbólicas
Tipo de entrada	Valores concretos	Variables simbólicas
Cobertura	Casos manuales	Todos los caminos posibles
Detección de errores	Limitada	Automática
Esfuerzo humano	Alto	Bajo
Valores de borde	Manuales	Generados automáticamente

3 Funcionamiento Básico de una Prueba Simbólica

La ejecución simbólica reemplaza los valores reales del programa por variables simbólicas. Estas variables representan cualquier posible valor de entrada.

Por ejemplo:

- $x \rightarrow \alpha$
- $y \rightarrow \beta$

Si el programa realiza operaciones sobre dichas variables, el sistema genera expresiones simbólicas y restricciones lógicas que describen el comportamiento del programa.

Obtención de condiciones de camino

Cuando el programa encuentra una condición lógica, la ejecución simbólica divide el análisis en diferentes caminos posibles.

Por ejemplo:

$\text{if } (x > 0)$

genera dos caminos:

- $\alpha > 0$
- $\alpha \leq 0$

Cada ruta acumula restricciones llamadas condiciones de camino (Path Conditions o PC). Estas condiciones permiten determinar qué valores concretos ejecutan cada parte del programa.

Características:

- Detecta errores automáticamente.
- Genera casos de prueba reproducibles.
- Compatible con flujos CI/CD.

3.3 Árbol de caminos

Raíz (PC: $\top$)

```

├──  $x > 0$  (PC:  $\alpha > 0$ )
|   ├──  $y > 5$  (PC:  $\alpha > 0 \wedge \beta > 5$ ) → camino A
|   └──  $y \leq 5$  (PC:  $\alpha > 0 \wedge \beta \leq 5$ ) → camino B
└──  $x \leq 0$  (PC:  $\alpha \leq 0$ ) → camino C
  
```

En este árbol:

- PC significa Path Condition.
- α representa la variable simbólica de x .
- β representa la variable simbólica de y .

3.4 Solvers SMT

Los solvers SMT (Satisfiability Modulo Theories) son herramientas que resuelven restricciones lógicas y matemáticas. Su función principal es determinar si una condición de camino puede satisfacerse y generar valores concretos para ejecutar esa ruta. Uno de los solvers más utilizados es Z3, desarrollado por Microsoft Research.

5. Herramientas que Implementan Pruebas Simbólicas

5.1 KLEE

KLEE es una herramienta de ejecución simbólica basada en LLVM utilizada principalmente para programas escritos en C y C++.

Características

- Genera automáticamente casos de prueba.
- Detecta errores de memoria.
- Aumenta la cobertura de pruebas.
- Compatible con integración continua.

5.2 CBMC

CBMC (C Bounded Model Checker) realiza verificación formal sobre programas en C, C++ y Java.

Características

- Verifica propiedades lógicas.
- Detecta desbordamientos y errores de memoria.
- Muy utilizado en sistemas embebidos.

5.3 angr

angr es una plataforma de análisis binario que utiliza ejecución simbólica para analizar programas compilados.

Características

- Análisis de binarios.
- Detección de vulnerabilidades.
- Integración con el solver Z3.

5.4 Manticore

Manticore es una herramienta enfocada en análisis de contratos inteligentes y seguridad.

Características

- Análisis de smart contracts.
- Detección de vulnerabilidades.
- Soporte para Ethereum.

4 Reflexiones Finales

Las pruebas simbólicas representan un cambio importante en la verificación de software, ya que permiten analizar sistemáticamente los caminos de ejecución de un programa y generar automáticamente casos de prueba críticos.

Aunque presentan limitaciones relacionadas con la complejidad computacional y la explosión de caminos, son una herramienta valiosa para mejorar la calidad y confiabilidad del software moderno.

5 Ejemplos

5.1 Ejemplo 1 – Análisis de Caminos Simbólicos

Código

```
function evaluar(x, y):
    if (x > 0):
        if (y > 5):
            return "A"
        else:
            return "B"
    else:
        return "C"
```

5.1.1 Reemplazo por variables simbólicas

- $x \rightarrow \alpha$
- $y \rightarrow \beta$

5.1.2 Condiciones de camino

Camino	Condición de camino	Salida
1	$\alpha > 0 \wedge \beta > 5$	A
2	$\alpha > 0 \wedge \beta \leq 5$	B

3	$\alpha \leq 0$	C
---	-----------------	---

5.1.3 Valores concretos de prueba

Camino	Valores posibles
A	$x = 2, y = 8$
B	$x = 3, y = 4$
C	$x = -1, y = 7$

5.2 Ejemplo 2 – Aplicación en Código Python

```
def calcular_descuento(precio, cliente_vip):
    if precio > 100:
        if cliente_vip:
            return precio * 0.8
        else:
            return precio * 0.9
    else:
        return precio
```

5.1.4 Variables simbólicas

- $\text{precio} \rightarrow \pi$
- $\text{cliente_vip} \rightarrow v$

5.1.5 Caminos identificados

Camino	Condición	Resultado
A	$\pi > 100 \wedge v = \text{True}$	$\text{precio} \times 0.8$
B	$\pi > 100 \wedge v = \text{False}$	$\text{precio} \times 0.9$

C	$\pi \leq 100$	precio
---	----------------	--------

Análisis de una condición incorrecta

Si la condición:

if precio > 100

se reemplaza por:

if precio >= 100

el comportamiento del programa cambiaría para el valor precio = 100.

Resultado original

- precio = 100 → no aplica descuento.

Resultado modificado

- precio = 100 → sí aplica descuento.

Esto demuestra cómo un pequeño cambio en una condición puede modificar completamente el comportamiento lógico del sistema y generar errores funcionales si no se realizan pruebas adecuadas.

6 Reflexiones Finales

Las pruebas simbólicas representan una técnica importante dentro de la verificación y validación de software, ya que permiten analizar automáticamente diferentes rutas de ejecución utilizando variables simbólicas y restricciones lógicas.

A diferencia de las pruebas tradicionales, que dependen de datos definidos manualmente, la ejecución simbólica facilita la generación automática de casos de prueba y mejora la detección de errores complejos o poco frecuentes. Aunque existen limitaciones relacionadas con la explosión de caminos y el costo computacional, esta técnica resulta especialmente útil en sistemas críticos, financieros y de seguridad, donde la confiabilidad del software es fundamental.

En conclusión, el razonamiento simbólico permite aumentar la cobertura de pruebas y mejorar la calidad del software moderno mediante un análisis más profundo y sistemático del comportamiento de los programas.

7 Referencias Bibliográficas

Cadar, C., Dunbar, D., & Engler, D. (2008). *KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs*.

Clarke, E., Kroening, D., & Lerda, F. (2004). *A tool for checking ANSI-C programs*.

King, J. C. (1976). *Symbolic execution and program testing*.

Baldoni, R., Coppa, E., D'Elia, D., Demetrescu, C., & Finocchi, I. (2018). *A survey of symbolic execution techniques*.

de Moura, L., & Bjørner, N. (2008). *Z3: An efficient SMT solver*.